

UHC Agentic Claims-Audit

Bare-Minimum VM Deployment Guide

Deploy the full UHC Agentic Claims-Audit stack on a single VM (Ubuntu 22.04/24.04 or Windows Server). Covers Django, Node claims-corebackend, databases, Celery, and corporate network access for UHC and Optum.

Deploy the UHC Agentic Claims-Audit stack on a single VM (Ubuntu 22.04/24.04 or Windows Server). This covers the Django agentic backend, the Node claims-corebackend relay, required databases, and optional Celery worker.

What runs on the VM

```
Browser / SPA
?
?
claims-corebackend (Node, port 4000) ? auth, user admin, proxy to Django
?
?
uhc-backend-v2      (Django, port 8000) ? builder, ingest, execute, agent-tools
?
??? Celery worker (same Python env) ? SOP ingestion + batch execution jobs
?
??? Data stores (local Docker or remote)
    PostgreSQL · Redis · Neo4j · MongoDB · RabbitMQ
| Component | Repo / path | Runtime | Default port |
|-----|-----|-----|-----|
| Django API | `uhc-backend-v2` (this repo) | Python **3.13** | 8000 |
| Celery worker | same repo | Python **3.13** | ? |
| Node relay | `claims-corebackend` | Node **20** | 4000 |
| React SPA (optional) | `claims-frontend` | Node 20 + yarn | 5173 |
```

Minimum VM size

```
| Resource | Bare minimum | Notes |
|-----|-----|-----|
| vCPU | 4 | SOP ingestion spawns CPU-heavy subprocesses |
| RAM | 8 GB | 16 GB recommended if ingestion + execution run concurrently |
| Disk | 50 GB | Logs, pip/node_modules, Mongo/Neo4j data |
| OS | Ubuntu 22.04/24.04 LTS **or** Windows Server 2019+ | Linux preferred for production |
```

Open inbound ports only as needed: 4000 (Node), 8000 (Django, internal), 5173 (dev SPA). Put nginx/IIS in front for TLS in production.

Inbound port requirements

```
| Port | Service | Exposure | Notes |
|-----|-----|-----|-----|
| 4000 | Node corebackend | Public / load balancer | Primary API endpoint for the SPA and external clients |
| 8000 | Django API | Internal only (localhost) | Never expose directly; proxied by Node |
| 5173 | React SPA (dev) | Dev only | Vite dev server; use `yarn build` + nginx in production |
| 443 / 80 | nginx / IIS (TLS) | Public | Terminate TLS here; reverse-proxy to Node :4000 |
| 22 | SSH / SCP | Corporate networks | Required for SCP from Optum image laptop (see below) |
| 5432 | PostgreSQL | Internal only | Bind to 127.0.0.1; never expose publicly |
| 6379 | Redis | Internal only | No auth by default; keep on localhost |
```

7474 / 7687	Neo4j HTTP / Bolt	Internal only	7474 = browser UI, 7687 = Bolt driver
27017	MongoDB	Internal only	Bind to 127.0.0.1
5672 / 15672	RabbitMQ / mgmt UI	Internal only	15672 = management console; firewall from public

Disk layout (recommended)

Path	Contents	Approx. size
`/opt/uhc/`	App code, Python venv, node_modules	~5 GB
`/var/lib/docker/`	PostgreSQL, Redis, Neo4j, Mongo, RabbitMQ data volumes	~20 GB
`/var/log/uhc/`	Gunicorn, Celery, Node access/error logs	~2 GB (rotate weekly)
`/tmp/`	Temporary ingestion artifacts	~5 GB burst

Network architecture

Recommended single-VM traffic flow:

Internet / corporate clients ? 443 (nginx TLS termination)
 nginx ? :4000 Node corebackend (auth, user admin, proxy)
 Node ? :8000 Django API (builder, ingest, execute, agent tools)
 Django ? :5432 PostgreSQL | :6379 Redis | :7687 Neo4j
 Django ? :27017 MongoDB | :5672 RabbitMQ
 Celery ? :5672 RabbitMQ (consume) ? :5432 PostgreSQL / :7687 Neo4j

Scaling considerations

Bottleneck	Symptom	Mitigation
SOP ingestion CPU	Gunicorn workers time out during ingest	Increase vCPU; raise gunicorn `--timeout`; scale Celery concurrency
LLM API latency	Slow ingest/execute responses	Cache results in Redis; queue via RabbitMQ
PostgreSQL connections	Too many clients error	Add pgBouncer in front of Postgres
Disk I/O	Slow Mongo/Neo4j queries	Move data volumes to dedicated SSD or managed DB
Memory pressure	OOM kills on Celery worker	Increase RAM to 16 GB; limit Celery `--concurrency=1`

Corporate network and access requirements (UHC / Optum)

The deployment VM must be reachable from all UHC and Optum corporate subnets and networks that will use or administer the platform. Work with network/security teams during provisioning ? do not treat this VM as an isolated lab host.

Cross-network reachability

Requirement	Detail
****UHC subnets****	Allow inbound and outbound traffic between the VM and all UHC VLANs/subnets that host auditors, admins, CI/CD, and dependent services.
****Optum subnets****	Same for Optum-side networks (including partner/VPN ranges used by Optum operations).
****Routing****	Ensure corporate DNS resolves the VM hostname from both UHC and Optum; no split-horizon or firewall rule should block one org from reaching the other via this host.
****Application access****	Browser/API clients on UHC and Optum networks must reach the public endpoint (nginx ****443**** or Node ****4000**** behind the load balancer).
****Internal DB ports****	PostgreSQL, Redis, Neo4j, MongoDB, and RabbitMQ remain ****internal only**** ? reachable from the VM and approved admin jump hosts, not from general corporate desktops.

SCP from Optum image laptop

Operators must be able to copy deployment artifacts from an Optum hardened (image) laptop to this VM using

SCP (or SFTP over the same SSH session).

Network / firewall (request from infra team):

Direction	Protocol	Port	Purpose
Optum image laptop ? VM	TCP	**22**	SSH / SCP file transfer
VM ? Optum/UHC subnets	TCP	**443**, **4000**	Health checks and API smoke tests from corporate clients

VM preparation (Ubuntu):

```
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
# Dedicated deploy user (recommended)
sudo useradd -m -s /bin/bash uhcdploy
sudo usermod -aG sudo uhcdploy
sudo mkdir -p /home/uhcdploy/.ssh && sudo chmod 700 /home/uhcdploy/.ssh
# Paste Optum laptop public key into authorized_keys
sudo nano /home/uhcdploy/.ssh/authorized_keys
sudo chown -R uhcdploy:uhcdploy /home/uhcdploy/.ssh
sudo chmod 600 /home/uhcdploy/.ssh/authorized_keys
```

SCP example (from Optum image laptop):

```
# Copy a release bundle or .env template to the VM
scp -r ./uhc-release-bundle uhcdploy@<vm-hostname-or-ip>:/opt/uhc/

# Single file
scp ./uhc-backend-v2/.env uhcdploy@<vm-hostname-or-ip>:/opt/uhc/uhc-backend-v2/.env
```

Verification checklist:

- [] Ping or `traceroute` from Optum image laptop to VM IP succeeds (if ICMP allowed).
- [] `ssh uhcdploy@<vm-ip>` succeeds from Optum image laptop.
- [] `scp` upload and download both work (test with a small file).
- [] Same SSH/SCP path works from at least one UHC admin workstation subnet.
- [] Corporate proxy/VPN does not strip or block port 22 to the VM segment.

Use key-based auth only; disable password authentication for SSH in production (PasswordAuthentication no in sshd_config).

Required databases

All five are used by the Django backend. Node needs PostgreSQL only (separate database on the same server is fine).

Service	Version	Used by	Env vars (Django)
PostgreSQL	16	Django (canonical data) + Node Prisma (`User` table)	`PG_HOST`, `PG_PORT`, `PG_USER`, `PG_PASSWORD`, `PG_DATABASE`
Redis	7	Cache, Celery result backend	`REDIS_HOST`, `REDIS_PORT`, `REDIS_USER`, `REDIS_PASSWORD`
Neo4j	5	SOP knowledge graph	`NEO4J_HOST`, `NEO4J_PORT`, `NEO4J_USER`, `NEO4J_PASSWORD`, `NEO4J_DATABASE`
MongoDB	7	Raw/parsed document snapshots	`MONGO_HOST`, `MONGO_PORT`, `MONGO_USER`, `MONGO_PASSWORD`, `MONGO_DATABASE`
RabbitMQ	3.13	Celery broker	`RABBITMQ_HOST`, `RABBITMQ_PORT`, `RABBITMQ_USER`, `RABBITMQ_PASSWORD`

Node corebackend uses one additional Postgres database via Prisma:

```
DATABASE_URL=postgresql://postgres:<password>@localhost:5432/claims_corebackend?schema=public
```

Use two databases on one Postgres instance for the smallest footprint:

```
- `agentic_flow_v2db` ? Django (`PG_DATABASE`)
```

- `claims\_corebackend` ? Node (`DATABASE\_URL`)

Quick start: databases with Docker (Ubuntu or Windows with Docker Desktop)

```
docker run -d --name pg      -p 5432:5432 -e POSTGRES_PASSWORD=postgres postgres:16
docker run -d --name redis   -p 6379:6379 redis:7
docker run -d --name neo4j    -p 7474:7474 -p 7687:7687 -e NEO4J_AUTH=neo4j/test1234 neo4j:5
docker run -d --name mongo    -p 27017:27017 mongo:7
docker run -d --name rmq      -p 5672:5672 -p 15672:15672 rabbitmq:3.13-management
```

Create the two Postgres databases:

```
docker exec -it pg psql -U postgres -c "CREATE DATABASE agentic_flow_v2db;"
docker exec -it pg psql -U postgres -c "CREATE DATABASE claims_corebackend;"
```

Ubuntu deployment

1. System packages

```
sudo apt update
sudo apt install -y git curl build-essential libpq-dev python3.13 python3.13-venv
# Node 20 (NodeSource)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo npm install -g yarn
```

Install Docker if you run databases locally: <https://docs.docker.com/engine/install/ubuntu/>

2. Clone repos

```
git clone <uhc-backend-v2-url> /opt/uhc/uhc-backend-v2
git clone <claims-corebackend-url> /opt/uhc/claims-corebackend
# optional frontend
git clone <claims-frontend-url> /opt/uhc/claims-frontend
```

3. Python backend

```
cd /opt/uhc/uhc-backend-v2
python3.13 -m venv /opt/uhc/venv
source /opt/uhc/venv/bin/activate
pip install -U pip
pip install -r requirements.txt
pip install -e uhc-sop-ingestion uhc-api-agent uhc-execution-engine
```

Create `/opt/uhc/uhc-backend-v2/.env` (see Environment variables below).

```
cd /opt/uhc/uhc-backend-v2
PYTHONPATH=. python manage.py migrate
PYTHONPATH=. python manage.py seed_builder_catalog
```

Run Django (dev):

```
PYTHONPATH=. python manage.py runserver 0.0.0.0:8000
```

Run Django (production-ish):

```
PYTHONPATH=. gunicorn sop_backend.wsgi:application --bind 0.0.0.0:8000 --workers 4 --timeout 120
```

Celery worker (separate terminal / systemd unit):

```
cd /opt/uhc/uhc-backend-v2
source /opt/uhc/venv/bin/activate
PYTHONPATH=. celery -A sop_backend worker -Q job_queue,celery --concurrency=1 -l INFO
```

4. Node corebackend

```
cd /opt/uhc/claims-corebackend
cp .env.example .env
# Edit .env: DATABASE_URL, JWT_SECRET (must match Django), DJANGO_BASE_URL=http://127.0.0.1:8000
yarn install
npx prisma generate
npx prisma migrate deploy
yarn build
yarn start          # production: node dist/server.js
# or: yarn dev      # development with hot reload
```

5. Optional frontend

```
cd /opt/uhc/claims-frontend
yarn install
# .env: VITE_API_BASE_URL=http://<vm-ip>:4000 (or your nginx URL)
yarn build && yarn preview # or yarn dev for development
```

6. systemd units (Ubuntu, optional)

Example Django unit /etc/systemd/system/uhc-django.service:

```
[Unit]
Description=UHC Django API
After=network.target docker.service

[Service]
User=ubuntu
WorkingDirectory=/opt/uhc/uhc-backend-v2
Environment=PYTHONPATH=.
EnvironmentFile=/opt/uhc/uhc-backend-v2/.env
ExecStart=/opt/uhc/venv/bin/gunicorn sop_backend.wsgi:application --bind 127.0.0.1:8000 --workers 4
--timeout 120
Restart=always

[Install]
WantedBy=multi-user.target
```

Mirror the pattern for uhc-celery.service (ExecStart = celery worker) and uhc-corebackend.service (ExecStart = node dist/server.js in claims-corebackend).

Windows deployment

1. Prerequisites

Install via installers or winget:

```
| Tool | Version |
|-----|-----|
| Python | 3.13 |
| Node.js | 20 LTS |
| Git | latest |
| Docker Desktop | latest (for local DBs) |
| Visual C++ Build Tools | required for `psycpg2-binary` if wheels fail |
```

Enable WSL2 if you prefer running Linux containers and bash scripts above inside Ubuntu on WSL.

2. Python backend (PowerShell)

```
cd C:\uhc\uhc-backend-v2
py -3.13 -m venv C:\uhc\venv
C:\uhc\venv\Scripts\Activate.ps1
pip install -U pip
pip install -r requirements.txt
pip install -e uhc-sop-ingestion uhc-api-agent uhc-execution-engine
copy .env.example .env # create and edit .env manually if no example exists
$env:PYTHONPATH = "."
python manage.py migrate
python manage.py seed_builder_catalog
python manage.py runserver 0.0.0.0:8000
```

Celery (second PowerShell window):

```
cd C:\uhc\uhc-backend-v2
C:\uhc\venv\Scripts\Activate.ps1
$env:PYTHONPATH = "."
celery -A sop_backend worker -Q job_queue,celery --concurrency=1 -l INFO
```

3. Node corebackend (PowerShell)

```
cd C:\uhc\claims-corebackend
copy .env.example .env
yarn install
npx prisma generate
npx prisma migrate deploy
yarn build
yarn start
```

4. Windows services

Use NSSM or PM2 for Windows to keep Django, Celery, and Node running across reboots. Point each service at the venv/python or node dist/server.js paths above.

Environment variables

Django (`uhc-backend-v2/.env`)

```
# Postgres (Django)
PG_HOST=localhost
PG_PORT=5432
PG_USER=postgres
PG_PASSWORD=postgres
PG_DATABASE=agentic_flow_v2db

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_USER=
REDIS_PASSWORD=

# Neo4j
NEO4J_HOST=localhost
NEO4J_PORT=7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=test1234
NEO4J_DATABASE=neo4j

# MongoDB
MONGO_HOST=localhost
MONGO_PORT=27017
MONGO_USER=
MONGO_PASSWORD=
MONGO_DATABASE=sop_ingestion

# RabbitMQ
RABBITMQ_HOST=localhost
RABBITMQ_PORT=5672
RABBITMQ_USER=guest
RABBITMQ_PASSWORD=guest

# Auth ? MUST match claims-corebackend JWT_SECRET (? 16 chars)
JWT_SECRET=change-me-to-a-32-char-or-longer-shared-secret

# Django
DJANGO_SECRET_KEY=change-me-in-production
DJANGO_DEBUG=false
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1,<vm-hostname-or-ip>
CORS_ORIGINS=http://localhost:5173,http://<vm-ip>:5173,http://<vm-ip>:4000

# LLM (required for SOP ingestion)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
```

```
# Pipeline tuning
MAX_DEPTH=4
MAX_DOCS=200
LLM_PROVIDER=anthropic
```

Node (`claims-corebackend/.env`)

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/claims_corebackend?schema=public
PORT=4000
NODE_ENV=production
DJANGO_BASE_URL=http://127.0.0.1:8000
JWT_SECRET=change-me-to-a-32-char-or-longer-shared-secret
JWT_EXPIRES_IN=7d
BOOTSTRAP_ADMIN=true
CORS_ORIGINS=http://localhost:5173,http://<vm-ip>:5173
```

Critical: JWT_SECRET must be identical on Node and Django.

Python packages (`requirements.txt` summary)

Core stack pulled by `pip install -r requirements.txt`:

Category	Packages
Web	Django 6, django-rest-framework, django-cors-headers, gunicorn
Task queue	celery, amqp, redis, kombu
Databases	psycopg2-binary, neo4j, pymongo, redis
LLM / agents	langgraph, langchain, langchain-anthropic, langchain-openai, anthropic, openai
Documents	beautifulsoup4, lxml, python-docx, openpyxl, pypdf
Auth	PyJWT
Editable sub-packages	`uhc-sop-ingestion`, `uhc-api-agent`, `uhc-execution-engine`

Requires git on the VM because editable installs reference GitHub subdirectories.

Node packages (`claims-corebackend/package.json`)

Dependency	Purpose
express	HTTP server
@prisma/client + prisma	Postgres ORM (User table)
jsonwebtoken	JWT minting (HS256)
bcryptjs	Password hashing
http-proxy-middleware	Forward `/api/builder/*`, `/api/ingest/*`, etc. to Django
cors, zod, winston	CORS, validation, logging

Scripts: `yarn dev` (tsx watch), `yarn build` + `yarn start` (production).

Health checks

```
# Django
curl http://localhost:8000/api/ingest/health/

# Node relay
curl http://localhost:4000/health

# Register first admin user (when BOOTSTRAP_ADMIN=true)
curl -X POST http://localhost:4000/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@example.com","password":"your-secure-password","name":"Admin"}'
```

Expected Django health: "celery": "ok" only when the Celery worker is running.

Troubleshooting

| Symptom | Fix |

|-----|-----|

| `JWT\_SECRET` env var missing or too short` on Django | Set `JWT_SECRET` ? 16 chars; match Node `.env` |

| `celery: "no\_workers"" in health check | Start Celery worker on `job_queue,celery` |

| `502` from Node on `/api/builder/\*` | Django not running or wrong `DJANGO\_BASE\_URL` |

| `MISSING ? No module named 'uhc\_sop\_ingestion` | `pip install -e uhc-sop-ingestion` and restart Django |

| Prisma migrate fails | Ensure `claims\_corebackend` database exists on Postgres |

| Ingestion fails on first LLM call | Set `OPENAI\_API\_KEY` and/or `ANTHROPIC\_API\_KEY` |

Production hardening (beyond bare minimum)

- Terminate TLS at **nginx** or **IIS**; proxy `/` ? Node :4000, keep Django on localhost only.
- Set `DJANGO\_DEBUG=false`, rotate `DJANGO\_SECRET\_KEY` and `JWT\_SECRET`.
- Run gunicorn behind nginx with adequate `--timeout` (120s+) for long ingest/execute calls.
- Use managed Postgres/Redis/etc. instead of Docker on the same VM when scaling.
- Restrict VM security group to required ports; never expose Neo4j/Mongo/Redis/RabbitMQ publicly.

For day-to-day developer setup, see SETUP.md.